

JITSU QA TEAM

New Member Training Document

Onboarding Guide for QA Engineers

WHAT YOU WILL LEARN

- ✓ Main business and how Jitsu makes money
- ✓ Warehouses, Regions and Zones — the geographic hierarchy
 - ✓ 4 Client Service Types and their delivery flows
- ✓ 4 Driver types (IC / DSP / 3P / Linehaul) and how each gets routes
 - ✓ Full URLs for 13 systems across Staging / Beta / Production
- ✓ End-to-End testing workflow from shipment submission to delivery
- ✓ QA process for maintain tickets: check real data + End User viewpoint
 - ✓ Critical rules and restrictions when testing on Production

Source: Confluence ENG + QA Team corrections

[Training Document — Confluence](#)

Version 2 — Updated May 2026

Table of Contents

Use this document as your roadmap. Each section builds on the previous one — start from the top.

- 1.** About Jitsu — Main Business
- 2.** Key Terminology & Concepts
- 3.** Warehouses, Regions & Zones
- 4.** Client Service Types & Delivery Flows
- 5.** Driver Types & How They Get Routes
- 6.** System Overview & Environment URLs
- 7.** Main Functions of the Platform
- 8.** QA Process & Workflow
- 9.** Test Case Template & QMetric
- 10.** Important Rules for QA in Production
- 11.** End-to-End Testing Guide (Step by Step)
- 11.0** Database & Tools Access via StrongDM
- 11.1** Setup: Create a Client
- 11.2** Submit a Shipment from Client Web
- 11.3** Clone Shipment from Production
- 11.4** Route the Shipment (Routing Web App)
- 11.5** Sign Up & Approve a Driver
- 11.6** Create Advance Booking Session
- 11.7** Create Schedule / Direct Booking
- 11.8** Create DSP Account & DSP Driver
- 11.9** Scan All Shipments Automatically (Postman)
- 11.10** Test Dimensions (Warehouse App)
- 11.11** Test Labels
- 11.12** Test Weekly Payment System
- 11.13** Test Disposable Shipments
- 11.14** Simulate Background Check (Turn API)
- 12.** Client Web — Functional Reference
- 13.** Recipient Web — Functional Reference
- 14.** Quick Reference: Useful Links

1. About Jitsu — Main Business

Jitsu (formerly known as Axlehire) is a last-mile delivery and logistics platform. The platform connects three main parties:

Party	Role
Clients (Merchants)	Businesses that ship parcels (e.g. The Farmers Dog, Hungryroot, Whiplash). They submit shipments to Jitsu via web upload or API.
Drivers	IC, DSP, 3P drivers pick up parcels and deliver to recipients. Linehaul drivers handle first/middle mile transport. All managed via Driver App.
Recipients	End customers who receive parcels. They can track shipments, update addresses, leave delivery instructions, and provide feedback via the Recipient web.

Core Delivery Lifecycle

A typical Jitsu shipment flows through these stages:

1. Client submits a shipment (single via API, or batch via CSV upload from the Client web).
2. Shipment is geocoded and validated, then Routing engine finds shortest paths and produces Routes.
3. Routes/Assignments are made available to drivers — IC drivers book via Booking Session / Direct Booking, DSP/3P are assigned directly.
4. Driver picks up parcels at the warehouse (Inbound sort → Scan → Audit), then delivers.
5. During delivery: dimensions may be measured (Warehouse App), parcels are scanned, POD captured.
6. Recipient receives notifications and can interact via the Recipient web.
7. Driver is paid weekly through the payment system; client receives reports.

2. Key Terminology & Concepts

Master these definitions early — they appear in every ticket and every discussion.

Term	Definition
Shipment	A single parcel from a client to a recipient. Each has a geocoded delivery address.
Route	The optimized path generated by the Routing engine to deliver a set of shipments — shortest path between stops.
Assignment	The set of shipments grouped together AFTER routing has computed an optimal path. An Assignment = shipments + a route + driver-ready container that can be booked or assigned.
Zone	A small geographic sub-area within a Region used for IC booking and route generation.
Region	A larger geographic market (e.g. SFO, LAX). One region contains multiple zones. ONE region has only ONE client service type.
Stem miles	Distance from the deployment facility to the driver's first stop, and from the last stop back. Minimizing stem miles is a key optimization goal.
OTD	On-Time Delivery — percentage of shipments delivered within the promised window. A primary KPI.
POD	Proof of Delivery — photo (and sometimes signature) captured by the driver at the recipient's location.
SLA	Service Level Contract — the per-client contract that defines delivery speed commitments and other terms. SLA is per-client, NOT a generic speed tier.
Courier	Jitsu-specific term for DSP or 3P drivers. NEVER used for IC drivers.
Tour cost	Base pay for a route before any bonuses.
Sprinkling	Inserting a late-arriving shipment into an existing route that has capacity, rather than creating a new route.
Geofencing	Location-based gate — driver actions like check-in or delivery only succeed when physically within an expected coordinate area.

Region vs Zone — Visual

USA → States → Regions (e.g. SFO, LAX) → Zones (sub-areas inside each region). Each region has its OWN zones — zones do not cross regions.

3. Warehouses, Regions & Zones

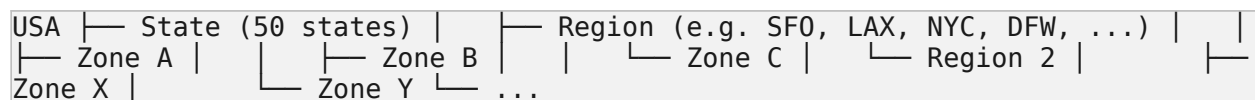
3.1 Warehouse Types

Jitsu uses 4 distinct types of warehouses. Knowing which type is involved in a flow tells you what test data to set up.

Warehouse Type	Purpose
Main Warehouse	Jitsu's main fixed facility — full sortation capability (inbound, sort, audit, outbound). The hub of every region. Starting point for large routes.
Mobile Hub	Lightweight, month-to-month rented mini-warehouse located near a high-density delivery cluster. Reduces stem miles. Can be opened or closed dynamically — sometimes as late as the day before delivery.
Client Warehouse	Warehouse owned by the Client. Used as pickup location for On-Demand, Specialty, and FCTD flows (Jitsu drivers come to Client WH to pick up).
DSP Warehouse	Warehouse owned by a DSP partner. Used as pickup point for DSP-operated routes.

3.2 Region & Zone Hierarchy

Jitsu operates in approximately 29 US markets. The geographic hierarchy is:



Critical rules about regions and zones

- USA is divided into states; each state can have multiple Regions.
- Each Region is divided into multiple Zones; zones do NOT cross regions.
- **ONE Region has only ONE Client Service Type.** (See Section 4.)
- IC drivers are restricted by Zone — they typically book within specific zones.
- DSP/3P drivers are NOT bound by zone boundaries — they operate on negotiated geographic areas.

 NOTE for QA setup

When you create a test client, you must add a `client_service` entry for each region the client serves. Since `region = single` service type, you can only assign a client to regions matching its type.

4. Client Service Types & Delivery Flows

Jitsu supports 4 client service types. When onboarding a new client, the Sales team negotiates which type the client falls into — this determines pricing and flow.

4.1 The 4 Service Types (by enum code)

In the database (field `logistic_type`), the enum codes are:

```
NEXT_DAYS(0), ON_DEMAND(1), SPECIALTY(2), FCTD(3);
```

NEXT_DAYS = Commingle (in business language)

Older docs use 'COMMINGLE' but the actual enum code 0 is NEXT_DAYS. In the team they may still be called 'Commingle' (business term) but always NEXT_DAYS in code and DB.

Type	Code	Description
NEXT_DAYS (Commingle)	0	Standard ship. Client delivers boxes to Jitsu Warehouse first; Jitsu sorts and delivers. Delivery window typically 8 AM – 8 PM. Like normal e-commerce ship.
ON_DEMAND	1	Rapid / same-day. Jitsu picks up from Client Warehouse/Store and delivers directly. Speed-priority. Higher pricing.
SPECIALTY	2	Special handling (cold chain, fragile, oversized). Pickup from Client. May require specific vehicle / certified driver.
FCTD	3	First-Class Transit Delivery — dedicated routes for a single client (not commingled with others). Contract-priced.

4.2 Delivery Flow per Type

NEXT_DAYS (Commingle)

```
Client → Jitsu Warehouse → Customer
```

- Client ships their boxes to Jitsu's Main Warehouse.
- Jitsu sorts, routes, and delivers to customer.
- Delivery window: 8 AM – 8 PM (standard ship).
- Always passes through Inbound Sort at Jitsu WH.

ON_DEMAND / SPECIALTY / FCTD

Client Warehouse / Store → Customer (typical, direct flow)
 OR Client Warehouse → Jitsu Warehouse → Customer (when sort/cross-dock needed)

- Jitsu drivers pick up from Client's warehouse/store.
- Typically deliver directly to customer (faster than Commingle).
- Higher pricing — same-day or contractual SLA.
- **Can** also go through Jitsu Warehouse for sort if operationally needed — not all bypass.

4.3 Mixing Client Types in Routes

Routes typically group shipments of the SAME client type — this is largely natural because:

- **ONE Region = ONE Client Service Type** → routes within a region automatically share the same type.
- Mixing can occur in edge cases (cross-region, special config) but is rare in practice.
- FCTD is the strictest — dedicated to one client by design.

Don't assume hard rules

Older docs said 'driver cannot book a route mixing different client types'. In practice this is mostly true because regions enforce single-type, but it's not an absolute code-level block for NEXT_DAYS / ON_DEMAND / SPECIALTY. Always verify the actual config when testing.

4.4 Quick Comparison

Criterion	NEXT_DAYS	ON_DEMAND	SPECIALTY	FCTD
Code	0	1	2	3
Pickup location	Jitsu Main WH	Client WH/Store	Client WH/Store	Client WH
Goes through Inbound Sort?	Always	Optional	Optional	Optional
Commingle with other clients?	Yes	Yes (rare)	Yes (rare)	Never (dedicated)
Speed	8AM–8PM	Same-day / rapid	Special	Per contract
Pricing	Lowest	High	Highest	Negotiated

Reference: [Client Service Type — Confluence](#)

4.5 Client Hierarchy & Brand Structure

Beyond service types, Jitsu supports 3 different ways a Client can be structured. A single business entity can have multiple sub-brands sharing infrastructure but with different visibility/billing rules.

Real example — Hello Fresh (Client ID 49)

Hello Fresh has all 3 client types under one umbrella — making it the canonical example of how Jitsu's client hierarchy works in practice.

The 3 Client Types

1 Parent Client

- Standalone main client with its own account.
- Has its own login credentials for the Client Web.
- Has its own unique `client_id`.
- Can submit shipments via both CSV upload and API.
- Independent billing — own `purchase_id`.

2 Brand (under Parent Client)

- Created FROM the Parent Client's account with a different brand name.
- Does NOT have its own login account for Client Web — shares with the Parent Client.
- Does NOT have its own `client_id` — uses Parent's ID.
- Distinguished only when submitting shipments — must include **shipment_code** in CSV or API payload to identify whether the shipment belongs to Parent or to a specific Brand.
- The `shipment_code` determines which brand name appears on the tracking page, label, reports, etc.
- Billing rolls up to Parent Client.

shipment_code is the key for Brand differentiation

When a CSV/API doesn't include `shipment_code`, the system treats all shipments as belonging to the Parent Client. To split shipments across brands, the `shipment_code` field **MUST** be present in every shipment record.

3 Brand but has `client_id` (independent login)

- Has its OWN distinct `client_id` (looks independent from Parent).
- CAN login to Client Web as a normal client — has its own account credentials.
- Still has `shipment_code` for differentiation when needed.
- BUT — **purchase_id is linked to the Parent Client** — meaning billing/invoicing still rolls up to the Parent.

- Use case: a larger sub-brand with its own ops team that needs independent access, while finance stays centralized at the Parent.

Quick Comparison

Attribute	Parent Client	Brand	Brand but has client_id
Own client_id?	✓ Yes	✗ No (shares Parent's)	✓ Yes (own ID)
Login Client Web?	✓ Yes — own account	✗ No — via Parent only	✓ Yes — own account
How to differentiate when submitting?	By client account/token	By shipment_code in CSV/API	By client account + shipment_code
purchase_id (billing)	Independent (own)	Rolls up to Parent	Linked to Parent's purchase_id
How DB distinguishes Brand vs Brand-with-id?	—	No own client row — only shipment_code mapping	Has own client row, but purchase_id field links to Parent

Implications for QA

- **When testing display logic** (tracking page, labels, reports) — always check the shipment_code field. The brand name shown to end users depends on it.
- **When testing billing/reports** — remember that purchase_id may be aggregated to the Parent even though shipments come from a Brand.
- **When setting up test data** — for Brand testing, you must include shipment_code in your CSV uploads or API payloads.
- **When testing Client Web login** — only Parent Clients and 'Brand with client_id' can login. Pure Brands cannot.
- If a ticket says 'shipment for Brand X', verify what type Brand X is in the DB before assuming the flow.

Production reference

Hello Fresh (Client ID 49) is the canonical example — it has Parent Client setup plus multiple Brands and at least one 'Brand with client_id'. When in doubt, check how Hello Fresh's brands are configured for reference.

5. Driver Types & How They Get Routes

Jitsu does not employ drivers directly. There are 4 distinct types — each with different rules for how they receive work.

5.1 The 4 Driver Types

Driver Type	Full Name	Role
IC	Independent Contractor	Individual drivers who book delivery opportunities through the Driver App. Restricted by zone. Last-mile only.
DSP	Delivery Service Provider	Contracted delivery companies covering negotiated areas. Drivers managed through their own DSP portal. Not bound by zones.
3P	Third-party gig platforms	Drivers from labor platforms (WorkWhile, InstaWork, Wonolo). Used as last resort when IC + DSP coverage is insufficient (peak season, surges).
Linehaul	Linehaul Driver	Transports first/middle mile freight: from Client to Jitsu warehouses, or between Jitsu warehouses. Moves pallets and truckloads (not individual parcels).

'Courier' terminology

In Jitsu, 'courier' refers ONLY to DSP and 3P drivers. IC drivers are NEVER called couriers.

5.2 How Each Driver Type Gets Routes

There are 3 main ways a driver receives a route — but not all driver types can use all methods:

Method 1: Assign route/ticket directly to driver

- A dispatcher or admin assigns a route/ticket to a specific driver — no booking required.

Method 2: Book route from Direct Booking

- Driver sees a route with full details (exact pay, location) and books it.

Method 3: Book ticket from Booking Session

- Advance booking — driver books tickets in a zone ahead of time, only seeing pay range and parcel count.

Critical permissions rule

Only IC drivers can book from Booking Session and Direct Booking. DSP and 3P drivers are assigned routes directly — either from their own DSP board, or by a Jitsu dispatcher.

5.3 Permission Matrix

Method	IC	DSP	3P	Linehaul
Booking Session (advance)	✓ Yes	✗ No	✗ No	—
Direct Booking	✓ Yes	✗ No	✗ No	—
Direct Assign by Jitsu Dispatcher	✓ Yes	✓ Yes	✓ Yes	—
Assign from DSP Board	✗ No	✓ Yes	✓ Yes	—

Linehaul Driver — quick note

Linehaul drivers operate on a completely separate workflow (NetSuite-driven auto-assignment, BOL-based pickup/dropoff). They do not participate in any of the booking methods above. Detailed Linehaul testing has its own dedicated documentation.

6. System Overview & Environment URLs

Jitsu has 3 environments — QA mainly works on Staging and Beta. Production testing follows strict rules (Section 10).

6.1 Environment Definitions

Environment	Purpose
Staging	Primary test environment. Full feature testing, free experimentation. Most QA work happens here.
Beta	Pre-production. Configuration must match Production. Used for final regression before release.
Production	Live system. Test only when necessary; always follow the strict rules in Section 10.

6.2 URL Pattern

URL pattern is consistent across systems:

Staging: <system>.staging.gojitsu.com Beta: <system>.beta.gojitsu.com Production: <system>.gojitsu.com

6.3 Web Apps — Full URL List

System	Purpose	Staging	Beta	Production
Dashboard (Admin)	Internal admin: clients, DSP, drivers, schedules	Open	Open	Open
Dispatch	Assignment management, routes, schedules, driver assignment	Open	Open	Open
Routing	Build routes from shipments (Routing problems)	Open	Open	Open
Client Web	Client-facing: submit shipments, view reports	Open	Open	Open
Recipient Web	End-customer tracking & delivery	Open	Open	Open

System	Purpose	Staging	Beta	Production
	management			
DSP Portal	DSP account management & DSP driver registration	Open	Open	Open
OAuth (Login)	SSO login gateway used by multiple systems	Open	Open	Open
Payment	Review IC/DSP payments, manage abnormal routes, payment formulas	Open	Open	Open
Geocoder Tool	Address validation, geocode management, geocode database	Open	Open	Open
Admin (account mgmt)	User account creation & management (separate from Dashboard)	Open	Open	Open
Inbound	Inbound sort operations, sort layout, monitor PPH	Open	Open	Open
Small Sort	Bin architecture for small parcel sorting	Open	Open	Open
ACL	Access Control List — manage permissions/roles	Open	Open	Open
Outbound Dashboard	Track expected/arriving drivers, parking, route staging	Open	Open	Open

**TIP**

Bookmark all Staging URLs first — that's where you'll spend most of your time. Beta URL = same pattern with .beta. Production = remove the .staging./.beta. part entirely.

6.4 Mobile Apps

Jitsu has 4 mobile apps for field operations. Mobile apps do not have web URLs — they release by version.

Mobile App	Purpose
Driver App (Jitsu Drive)	Used by IC, DSP, 3P drivers: registration, booking, pickup, delivery, POD, payment screens.
Outbound App	Used by Outbound staff: yard management, ID scan, route check-out to drivers.
Warehouse App	Used by warehouse staff: receive inbound containers, sort, scan, measure parcels (dimensions), audit.
Driver Linehaul App	Used by Linehaul drivers: BOL capture, first/middle mile transit. Separate from Jitsu Drive.

Mobile App Release & Testing Workflow

Mobile apps are released by version — testing is environment-staged, not URL-based:

8. On Staging — download builds from Firebase to test.
9. All tickets in a version must PASS on Staging.
10. Once all pass, dev pushes the build to TestFlight (the Production environment for mobile).
11. Test the build on TestFlight (Production).
12. If all tickets pass on Production → version is READY to release.

NOTE — Mobile testing scope

Mobile apps are tested ONLY on Staging and Production (TestFlight). There is no Beta build for mobile — the Beta environment is for web apps only.

7. Main Functions of the Platform

These are the core functional areas a QA must understand. Each one will eventually need test cases.

7.1 Booking & Assignment

Type	Description
Booking Session	Advance booking — IC drivers book routes in advance through a session window.
Direct Assign	Admin/Dispatcher assigns a driver directly to an assignment.
Direct Booking	IC driver books a route immediately from a schedule with full details.
Ticket Booking	IC driver books individual tickets (within Booking Session) instead of full routes.

7.2 Recipient

- Tracking home page — search by tracking code.
- Manage Delivery — only visible when shipment is not yet completed.
- Address questionnaire — recipient answers address-type questions (single house has different follow-up).
- Edit address.
- Codes (call-in, building, gate, etc.).
- Delivery instructions.
- Location Pins & Preferred Drop-off (upload photo to help driver find location).
- Feedback — view POD, like/dislike rating, send feedback, tips.

7.3 Inbound Sorting (Warehouse)

- Scan — incoming parcels are scanned at the warehouse.
- Audit — verify scanned shipments match expectations.
- Sprinkling — insert late shipments into existing routes with capacity.

7.4 Payment

- Weekly payment system for IC drivers and DSP drivers.
- Tested via the driver_payment worker on the Schedules page.
- Supports payment methods including BRANCH_APP, Direct Deposit, etc.

7.5 Driver App

- Registration (sign up + background check).
- Delivery follow: Ticket booking, ETA, Direct booking, Assigned route.
- Change stop.
- Tutorial flow.
- Payment (early payout, weekly payout).

7.6 DSP Driver

- DSP Account (created in Dashboard).
- DSP Driver Account (created from DSP Portal by DSP itself, approved in Dashboard).
- DSP Payment.

7.7 OB App / Web (Outbound)

- ID scan.
- OB check out — driver picks up shipments before leaving the warehouse.

7.8 Small Parcel & Shipment Operations

- Assignment: Create / Update / Adding-removing shipments from a route.
- Shipment: Cancel / Discard / Disposable.

8. QA Process & Workflow

Follow this process for every assigned ticket. Consistency is what keeps the team efficient.

GOLDEN RULE for Maintain Tickets

When you receive a maintain ticket (not a new feature), first check the current flow on Production and look at REAL DATA. Understand how end users actually behave today — then build test cases from the end-user viewpoint to extract maximum coverage from the ticket.

Step 1 — Understand the Ticket

- Read the user story and acceptance criteria carefully.
- Identify which part of the delivery flow it touches (Section 1.2).
- Identify which driver type / client type / warehouse is involved.

If it's a Maintain ticket:

13. Check the current flow on Production first.
14. Look at REAL data — what does end-user behavior actually look like?
15. Build the test case from an end-user point of view.
16. Goal: extract maximum coverage by exploring the ticket from real usage patterns.

Step 2 — Write Test Cases

- Write test cases using the Confluence-attached format (see Section 9).
- Upload them to QMetry under the Test Case tab.
- Test cases are organized by Application.

Folder structure in QMetry:

17. Navigate to the correct Application.
18. Find the relevant function.
19. Create a folder named with the ticket ID (e.g. ENG-6799).
20. Import the prepared test case file, or create test cases directly.

Step 3 — Test Cycle

For tickets that have test cases, testing must be executed through Test Cycles in each environment. Sprints or app versions are already created in the Test Cycle tab.

Web App tickets (tested by Sprint):

21. Navigate to the correct Sprint.
22. Create a new folder named with the ticket ID.
23. Inside the folder, create test cycles for each environment.

Naming convention:

Folder name: Ticket_ID Cycle name: [Environment]_[Ticket_ID] Ticket name

Example:

Folder: ENG-6799 Cycle (Staging): [Staging]_[BE] Update API to support selection of reason codes for unbooked tickets

Step 4 — Verification Comments on Jira**⚠ IMPORTANT**

When verifying on Staging, make sure the configuration in Staging matches Production. On Beta, the config should also match Prod. During testing, watch the Datadog logs to make sure no errors appear after a new deploy.

Every Jira ticket needs a clear verification comment containing:

- Tested environment(s).
- Test result: PASSED or FAILED.
- Clear explanation of the result.
- Evidence attached (main flow is sufficient).

Example comment:

Tested on Staging – PASSED

If a ticket cannot be tested on Beta or Production, clearly state the reason in the comment.

Step 5 — Tickets Without Test Cases

For tickets that do not require formal test cases (verified via a checklist):

- Clearly list all verified scenarios in the Jira comment.

Step 6 — Documentation

For tickets that introduce new configuration or logic without existing documentation:

- QA is encouraged to create the documentation.

- AI tools may be used to assist with writing.

Step 7 — Reporting

Daily QA Report

- Update the QA tracking file (Excel) daily.
- Post updates in the QA channel so the lead can track progress.
- If unable to update the file daily, proactively report to the QA lead.

Mobile Testing — Version Completion Report

- Actively track the progress of the app version.
- Once all assigned tickets pass on Staging → notify the qa-mobile channel and request push to TestFlight (Production).
- Once all pass on Production → version is READY to release.

Full reference: [QA Process — Confluence](#)

Step 8 — Bug Reporting

When testing reveals an issue, log a bug ticket on Jira. Jira has pre-defined fields — fill them according to the template below.

Required Fields on Jira

Field	Required?	Content to fill
Title (Summary)	✓	Format: [Component/Feature] - [Actual issue]
Environment	✓	Staging / Beta / Production
Description	✓	1-2 câu mô tả ngắn gọn hiện tượng
Preconditions	✓	Config flags + test data + role cần dùng
Steps to Reproduce	✓	Steps cụ thể, dev follow phải reproduce được 100%
Actual Result	✓	Hiện tượng SAI — mô tả objective, không phán đoán nguyên nhân
Expected Result	✓	Reference AC hoặc business logic
Evidence	✓	Screenshot/Video/Datadog log/Test data IDs. Không có evidence = ticket bị bounce
Issue Type	✓	Phải chọn Bug
Priority	✓	Theo business impact (Critical/High/Medium/Low)
Assignee (Dev)	⚠ Optional	Dev ownership component (nếu biết)
QA Review	⚠ Optional với rule	Xem rule QA Review bên dưới

🎯 Rule QUAN TRỌNG về QA Review field

Case 1 — Log bug RIÊNG (standalone): QA Review = Optional, để trống cho PO/Lead assign sau. Case 2 — Log bug TRONG QUÁ TRÌNH TEST ticket mình đang được assign: QA Review = chính bản thân QA log bug đó (BẮT BUỘC). Lý do: bạn đang test ticket A → tìm bug → log bug B. Khi dev fix bug B xong, chính bạn verify vì hiểu context ticket A.

Bug MOBILE — Thêm fields bắt buộc

Khi log bug cho mobile apps (Driver / Outbound / Warehouse / Linehaul), điền thêm:

App & Version: - App: [Driver App / Outbound App / Warehouse App / Linehaul App] - Current version: [vd 2.8.2, version bug được phát hiện] - Build source: [Firebase Staging build / TestFlight Prod / Play Store] Fix Version: [Version mà bug này sẽ được fix, vd: 2.9.0]

Tại sao Mobile cần các fields này?

- App + Current Version: Bug có thể chỉ xảy ra trên 1 version cụ thể (VD: MOB-2643 chỉ ở v2.8.0). Dev cần biết version để reproduce và xác định regression.
- Build source: Phân biệt Staging Firebase vs TestFlight (Prod) — có thể có behavior khác.
- Fix Version (**critical**): Version target sẽ release fix. Track release roadmap, mobile QA biết version nào cần test lại, mobile có cycle release riêng theo version.

Bảng so sánh: Bug Mobile vs Bug Web

Aspect	Bug Mobile	Bug Web
App identifier	App name + Current Version + Build source	Browser + Version (note vào Evidence)
Environment	Staging Firebase / TestFlight Prod	Staging / Beta / Prod URL
Fix Version	✅ Required (track release cycle)	❌ Không cần (web deploy continuous)
Evidence ưu tiên	Screen recording + Datadog log	Console errors + Network logs (DevTools F12)
User identity	(đã có trong test data IDs)	Username + Scopes (note vào Preconditions)

✓ Checklist trước khi submit Bug

📝 Nội dung

- Title clear format: [Component] - [Issue]
- Environment ghi rõ (Staging/Beta/Prod)
- Steps to Reproduce có thể follow để reproduce 100%
- Actual ≠ Expected — viết riêng, không gộp
- Evidence đính kèm (screenshot/video tối thiểu)

📱 Nếu Mobile

- App + Current version + Build source
- Fix Version (version sẽ release fix)

🎯 Jira fields

- Issue Type = Bug
- Priority đúng level
- Assignee = dev hoặc PO/Lead
- QA Review = chính bản thân nếu đang test ticket được assign, optional nếu standalone
- Linked Issues = link parent ticket nếu bug từ ticket đang test

📖 Real Example — Mobile Bug (theo MOB-2676)

Title: POD photo fallback upload via API stores file in S3 instead of GCS
 Environment: Staging Description: When `upload_to_firebase_storage` is ON or `upload_pod_to_firebase_storage` is ON, POD photo upload should use Firebase Storage/GCS. If the upload fails by event, the app falls back to uploading via API. However, the fallback upload stores the POD photo in S3 instead of GCS. App & Version: - App: Driver App (Jitsu Drive) - Current version: 2.8.2 - Build source: Firebase Staging build Fix Version: 2.9.0 Preconditions: - Config flag `upload_to_firebase_storage` = ON - Test driver booked + picked up a route - At least 1 shipment ready for delivery Steps to Reproduce: 1. Driver mở app, complete pickup route 2. Đến stop đầu tiên, tap "Arrived at Stop" 3. Tap "Take POD Photo" và chụp 4. Simulate event failure (block event upload endpoint) 5. App fallback sang API upload 6. Check storage location của POD photo Actual Result: POD photo được upload và lưu vào S3 (qua API fallback) Expected Result: Khi flag `upload_to_firebase_storage` hoặc `upload_pod_to_firebase_storage` = ON, fallback API cũng phải lưu POD vào GCS, không phải S3 Evidence: - Screen recording: [attach] - Datadog log: [link] - Test data: driver_id DR_7873, shipment SHIP12345 Jira fields: - Issue Type: Bug - Priority: Medium - Assignee: Vinh Truong (mobile dev) - QA Review: Linh Mau (đang test feature POD GCS migration) - Fix Version: 2.9.0 - Linked Issues: relates to MOB-2017

Real Example — Web Bug (theo ENA-1131)

Title: [Claims] Payout % or Payout Cap value not saved when only one field filled Environment: Staging Description: When creating a Claims Client Policy, if only Payout % or only Payout Cap is filled (not both), the policy is saved but the filled value is lost. User must fill both fields for either to persist. Preconditions: - Test client exists (client_id 3771) - User has admin scope to access Claims module - Login: qa.test@jitsu.com (scopes: admin, claims-admin) - Browser: Chrome 131 on macOS 14.2 Steps to Reproduce: 1. Login Dashboard với account claims-admin 2. Navigate Claims → Policy → New Policy 3. Fill Client ID = 3771 4. Fill ONLY "Payout %" = 50 (leave Payout Cap empty) 5. Click Save Actual Result: Policy được tạo nhưng field Payout % = null (value 50 bị mất) Expected Result: Policy được tạo với Payout % = 50, Payout Cap = null (không bắt buộc cả 2) Evidence: - Page URL: https://dashboard.staging.gojitsu.com/claims/policy/new - Screenshot trước/sau Save: [attach] - Console error: "Validation skipped for empty Payout Cap" - Network: PUT /claims/policies returns 200 but payload missing payout_percent - Test data: client_id 3771 Jira fields: - Issue Type: Bug - Priority: Medium - Assignee: [Web dev assigned to Claims] - QA Review: [Bạn - nếu đang test Claims feature] - Linked Issues: relates to ENA-1022

Common Mistakes — Tránh các sai lầm sau

24. Title mở hồ: "Bug ở Driver App" →  "Driver App crash khi tap POD trên Android"
25. Description = Steps to Reproduce → 2 fields KHÁC NHAU. Description = tóm tắt 1-2 câu.
26. Không có Expected Result → Dev không biết bug ở đâu. Luôn reference AC.
27. Phán đoán nguyên nhân: "Bug do API lỗi" → Mô tả hiện tượng, để dev điều tra root cause.
28. Không attach evidence → Ticket bị bounce ngay.
29. Skip Preconditions → Dev mất 30 phút tự setup.
30. Quên link parent ticket + quên set QA Review = chính mình khi bug từ ticket đang test.

Mobile-specific tip

Fix Version là field quan trọng nhất với mobile bug. Mobile release theo cycle version (vd v1.18.0, v2.8.2). Không có Fix Version → PO không biết bug lên release nào → không estimate được timeline cho client.

Web-specific tip

Web deploy continuous (không có version cycle như mobile) → không cần Fix Version. Thay vào đó, Console + Network logs là evidence mạnh nhất → paste vào Evidence field.

9. Test Case Template & QMetry

Jitsu uses a standard test case template (Google Sheet) and the QMetry Excel import format for uploading to QMetry.

Template Location

The official test case template: [TCs Template — Google Sheets](#)

Recommended Test Case Structure

Field	Notes
Test Case ID	Auto or manual; matches ticket ID prefix
Summary	One-sentence description of what is being tested
Precondition	Data/setup required before execution
Test Step	Clear, numbered actions
Expected Result	What should happen after each step
Priority	High / Medium / Low
Type	Acceptance / E2E / Regression / Smoke

10. Important Rules for QA in Production



CRITICAL — READ BEFORE TOUCHING PRODUCTION

These rules exist because mistakes in Production affect real merchants, real drivers, and real revenue. Re-read them every time before any Prod test session.

Area	Rule
Client name	Always add the prefix 'Test' (e.g. Test_Acme).
POD review	Add the client ID when adding shipments to the POD review.
Background check	DO NOT submit a real background check for test drivers.
Assignment label	Only create assignment labels named 'TEST' or 'QA'.
Branch account	DO NOT sign up for a Branch account in the Driver App on Prod.
Email	Only use a Jitsu (@jitsu / @gojitsu / @axlehire) email when creating accounts.
Driver payout	DO NOT trigger early payout in the Driver app.
Cleanup	Delete the assignment / Booking Session / Schedule / Inbound layout in Prod after testing is done.

Reference: [Important Note for QA team in Prod — Confluence](#)

11. End-to-End Testing Guide

This section walks you through a complete E2E flow. As a new QA, follow it in order at least once on Staging — that will give you a working mental model of how Jitsu actually moves a parcel from a client all the way to a recipient.

GOAL

By the end of this section, you should be able to: create a Client → submit a shipment → route it → assign a driver who books and delivers it → simulate payment.

E2E Flow Overview

1. Create a Client (Dashboard)
2. Submit a Shipment (Client web — batch or single API)
3. Route the Shipment (Routing web)
4. Sign Up Driver (Driver App + simulate FCRA approval)
5. Create Booking Session / Schedule
6. Driver books and delivers
7. Trigger Weekly Payment

11.0 Database & Tools Access via StrongDM

Reference: [StrongDM — Confluence](#)

IMPORTANT — Required before E2E testing

StrongDM is how QA accesses Jitsu's protected resources: databases (PostgreSQL, MongoDB, Cassandra), Jenkins, internal web apps, Consul, Nexus repositories, and SSH to servers. Setup this BEFORE starting the E2E flow.

What is StrongDM?

StrongDM is a proxy that manages and audits access to protected resources such as databases, servers, clusters, and web apps. It replaces traditional VPN setup. You won't need to remember resource addresses or credentials — StrongDM handles it all.

The local client tunnels requests from your workstation to the gateway, so you only need to connect to localhost.

Learn more: [How StrongDM Works](#)

Installation

StrongDM Desktop Client

31. Login to <https://app.strongdm.com/> and go to the Download & Install page.
32. Click 'Download StrongDM for macOS' (or your OS) to get the desktop app and CLI.
33. Install the application.
34. Open SDM client — login with your Jitsu credentials.

StrongDM CLI Client (Optional)

For developers/QA who prefer command-line workflow, install the 'sdm' CLI alongside the desktop app. CLI is required for SSH/SCP use cases.

Basic Usage — Connect to a Resource

35. Open SDM desktop client.
36. Find the resource you need (e.g. 'staging-postgresql', 'staging-mongodb').
37. Click the resource → wait until status shows ♥ Connected.
38. Now you can access the resource via localhost + corresponding port.



Resource access requests

If you don't see a resource you need (e.g. specific database, server, repository), submit a request in SDM. DevOps/IT must approve. Allow 1-2 business days.

Connect to Internal Websites (*.gojitsu.sdm.network)

Many Jitsu internal tools (Jenkins, Consul UI, Nexus, etc.) are accessible only through StrongDM proxy. Setup the system proxy once, then access them from your browser.

macOS — Configure Browser Proxy

39. Open System Preferences → Network.
40. Select your current connection (wireless or ethernet) → click Advanced.
41. Click the Proxies tab.
42. Check Automatic Proxy Configuration.
43. Add this URL to the URL field:

```
https://app.strongdm.com/proxy.pac
```

44. Click OK → Apply.
45. Open the internal website in your browser — it should now load successfully.

Use Case: SSH and SCP

For connecting to Jitsu servers via SSH/SCP (e.g. for debugging, log inspection).

Prerequisites

- Install sdm CLI (see Installation section above).

SSH to a server

```
sdm ssh <name_of_server>
```

SCP file transfer

Run this command FIRST to setup the SCP alias:

```
eval "$(sdm ssh alias | grep 'scp')"
```

Now you can use scp normally to/from Jitsu servers.

Use Case: Connect to PostgreSQL

Example: connect to Staging PostgreSQL using DBeaver, pgAdmin, or any PostgreSQL client.

Connection Settings

- **Host:** localhost
- **Port:** the port shown next to the resource in SDM client (varies per resource)
- **Database:** can be left empty (or specify the DB name like 'axlehire_dataorch')
- **Authentication:** can be omitted (SDM handles it automatically)



Troubleshooting — DBeaver shows no tables?

Enable 'Show all databases' option in the connection settings, OR specify the correct database name in the Database field. By default, DBeaver only shows the default database.

Use Case: Connect to MongoDB

Example: connect to Staging MongoDB using Studio 3T, MongoDB Compass, or any MongoDB client.

Connection Settings

- **Host:** localhost
- **Port:** the port shown next to the resource in SDM client
- **Authentication:** not required (SDM proxies the connection)

Common MongoDB collections you'll query:

- `sort_center` — sortation center configs (`enable_measure`, `dimension_config`)
- `driver_registration_record` — driver BG check status
- `measurement_obligations` — dimension records

Use Case: Connect to Cassandra

For accessing Cassandra cluster (event persistence, time-series data).

Detailed guide:

[Cassandra Access — Confluence](#)

Use Case: Access Jenkins

46. Ensure your browser proxy is configured (see 'Connect to Internal Websites' above).
47. Connect to Jenkins resource in SDM client (wait for ♥ Connected).
48. Open Jenkins in browser:

```
https://jenkins.gojitsu.sdm.network/
```

Use Case: Access Consul in Code

For accessing service discovery configs in development environments.

Step 1: Edit /etc/hosts

Add these lines to your /etc/hosts file:

```
# Alias for Consul local 127.0.0.1      consul-staging-  
local.gojitsu.sdm.network 127.0.0.1      consul-local.gojitsu.sdm.network  
127.0.0.1      consul-prod-rw-local.gojitsu.sdm.network
```

Step 2: Use the appropriate URL

For Staging:

```
https://consul-staging-local.gojitsu.sdm.network:14334
```

For RW Beta, RO Prod:

```
https://consul-local.gojitsu.sdm.network:14333
```

For RW Beta, RW Prod:

```
https://consul-prod-rw-local.gojitsu.sdm.network:14336
```

Use Case: Download Java Libraries from Nexus (Maven)

For Java projects requiring private Jitsu libraries from Nexus repository.

Configuration

Update your Maven settings file at `~/.m2/settings.xml` with the following configuration. This includes:

- SDM proxy setup (127.0.0.1:65230)
- Nexus repository URLs (snapshots, releases, public)
- Both staging and release profiles

Full settings.xml template

See the Confluence StrongDM page for the complete settings.xml file with all required proxies and repository configurations.

Use Case: NPM/Yarn Packages from Nexus

For Node.js / React projects requiring private @gojitsu packages.

Prerequisites

- Ensure 'Nexus Repository' resource is connected in StrongDM.

Configure ~/.npmrc

```
@gojitsu:registry=https://nexus.gojitsu.sdm.network/repository/npm-private/  
proxy=http://127.0.0.1:65230 https-proxy=http://127.0.0.1:65230  
noproxy[]=registry.npmjs.org noproxy[]=registry.yarnpkg.com  
noproxy[]=yarnpkg.com #always-auth=true
```

Upload Packages

```
npm publish --registry=https://nexus-rw.gojitsu.sdm.network/repository/npm-private/
```

Use Case: Repository for Dart and Flutter

For Flutter mobile development requiring Jitsu's private Dart packages.

Step 1: Request access

Request access to the 'Nexus Repository' resource in SDM client.

Step 2: Edit `/etc/hosts`

127.0.0.1	nexus.axlehire.com	127.0.0.1	nexus.gojitsu.com
-----------	--------------------	-----------	-------------------

Step 3: Use `pub` commands normally

You can now run 'pub get', 'flutter pub get', etc. in your Flutter projects — they'll download Jitsu private packages via the SDM proxy.

Common Troubleshooting

1. Resource shows 'Disconnected' or won't connect

- Make sure SDM Desktop Client is running and you're logged in.
- Restart the SDM client.
- Check your internet connection.
- Request DevOps to verify your permissions for that resource.

2. PostgreSQL DBeaver shows no tables

- Enable 'Show all databases' option in connection settings.
- Or specify the correct database name in the Database field.

3. Internal website (e.g. Jenkins) won't load

- Verify system proxy is configured correctly (<https://app.strongdm.com/proxy.pac>).
- Make sure the corresponding resource is ♥ Connected in SDM.
- Try clearing browser cache.

4. NPM/Maven downloads fail

- Verify Nexus resource is connected in SDM.
- Check that proxy port in your config matches the port shown in SDM.
- Restart the package manager.

11.0.5 Prerequisites — Create Admin Account

IMPORTANT — Required before Step 11.1

You **MUST** have an Admin account before doing any E2E step. The Admin account is created on the Admin App (admin.staging.gojitsu.com), NOT the Dashboard. With this admin account, you can login Dashboard and create Client accounts (Step 11.1).

Admin App vs Dashboard — clarification

New QAs often confuse these two systems. They are different:

System	URL	Purpose
Admin App	admin.staging.gojitsu.com	Create & manage user accounts (admin users, client accounts). Layer ABOVE Dashboard for account management.
Dashboard	dashboard.staging.gojitsu.com	Daily admin operations (manage clients, drivers, DSP, schedules, regions). REQUIRES an admin account from Admin App.

How to get an Admin Account

On Staging, QA Lead has Super-admin privilege and can:

- Create new Admin accounts for new QA members.
- Assign/update scopes (permissions) as needed.
- Reset password if forgotten.

New member workflow

On Day 1, message QA Lead to request your Staging Admin account. Provide your Jitsu email and desired scopes. QA Lead will create the account and share credentials securely (via 1Password or DM).

Common Scopes for QA

Scope	Permission
admin	General admin rights — enough for most QA work (create clients, view drivers, schedules)
super-admin	Highest level — can create users + modify permissions.

Scope	Permission
	Reserved for QA Lead/DevOps.
warehouse-manager	Manage warehouse, sort layout, inbound operations
lead-dispatcher	Split route, advanced dispatch operations
finance-admin	Manage payments, payouts, financial settings
finance-billings	Manage EDI invoices, billing operations

NOTE — Scope priority

For most QA test work (creating clients, running E2E), scope `admin` is sufficient. Only request additional scopes (finance-admin, lead-dispatcher) when working on tickets that specifically require them.

Steps (after QA Lead creates your account)

49. Receive Admin account credentials from QA Lead (via 1Password share or DM).
50. Open Admin App Staging: <https://admin.staging.gojitsu.com/>
51. Verify login works.
52. Open Dashboard Staging: <https://dashboard.staging.gojitsu.com/>
53. Login with the SAME credentials — if you see admin menu (Client, DSP, Region, Warehouse...) → setup is complete.

If Dashboard login fails

If you can login Admin App but not Dashboard, your account may be missing the required scopes. DM QA Lead to add `admin` scope to your account.

For Super-admin tasks (Staging only)

If you need to create another user account on Staging or update scopes for an existing user:

- Ask QA Lead (has super-admin on Staging).
- Provide details: target user email, scopes needed, reason.
- QA Lead can also help reset passwords if locked out.

Production note

On Production, super-admin privilege is restricted to DevOps/security team. QA Lead does NOT have super-admin on Production. For Production account requests, escalate to DevOps via #devops Slack channel.

11.1 Setup: Create a Client

Reference: [How to create Client — Confluence](#)

Precondition

You must have an Admin account that can login to Dashboard (see Section 11.0.5). Without it, you cannot create test clients.

Steps

54. Open the Dashboard (Staging): <https://dashboard.staging.gojitsu.com/>
55. Select Client in the left menu.
56. Click Add — the 'ADD NEW CLIENT' pop-up appears.
57. Enter all valid information; choose one of the 4 client service types (NEXT_DAYS / ON_DEMAND / SPECIALTY / FCTD).
58. After creation, add the client service directly into the client_service table in PostgreSQL.

NOTE — Region/Type matching

For each region you want the client to serve, you must add a corresponding entry into the client_service table. Since ONE region = ONE client service type, the region you select must match the client's type.

Decide the Client Setup Type First

Before creating a test client, decide which of the 3 client structures you need (see Section 4.5 for the theory). This affects what test data you'll prepare:

If creating a Parent Client (Type 1)

- Create the client normally on Dashboard with own credentials.
- Add client_service entries for each region it serves.
- No special fields needed — this is the standard flow.
- Useful for: testing standard shipment submission, login, billing flows.

If creating a Brand (Type 2 — no own client_id)

- Create the Parent Client first (if not already exists).
- Add the Brand under the Parent — usually configured at the brand/shipment level, not as a new client row.
- When submitting test shipments, **ALWAYS include shipment_code in CSV/API** — otherwise the system treats shipments as Parent's, not the Brand's.
- Verify display in: tracking page, label, reports — brand name must match shipment_code.
- Cannot login as the Brand — login uses Parent's credentials only.

If creating a Brand with own client_id (Type 3)

- Create the Brand as a separate client row in Dashboard (gets its own client_id).
- Configure its login credentials — it can login Client Web independently.
- In DB, set **purchase_id** to link back to the Parent Client (this is what makes billing roll up).
- Add client_service entries for regions this Brand serves.
- Test scenarios: login flow, shipment submission, but verify billing/reports aggregate at Parent level.



Best practice for QA test setup

For most tickets, a simple Parent Client (Type 1) is enough. Only set up Brands (Type 2 or 3) when the ticket specifically involves brand differentiation, multi-brand display logic, or billing aggregation testing.

Real reference in Production

To see how all 3 types coexist, look up Hello Fresh (Client ID 49) in Production:

- It has the Parent Client setup.
- Multiple Brands under it (Type 2).
- At least one Brand with own client_id (Type 3) — purchase_id traces back to client 49.
- Don't modify Hello Fresh data — just use it as reference for understanding the structure.

11.2 Submit a Shipment from Client Web

Reference: [How to submit a shipment from Client web](#)

Option A — Submit via Batch (CSV)

59. In Client web, click 'Upload Batch' — the 'Upload new shipment batch' pop-up appears.
60. Upload your CSV file and click Upload.
61. The uploaded file appears under the NEEDS REVIEW section.
62. Locate the uploaded batch and click Finalize.

Option B — Submit Single Shipment via API

- Use Postman with the Client token (public token) to call the Submit Shipment API.
- Pick the Staging environment in Postman.

11.3 Clone Shipment from Production

Useful when you need realistic data shapes in Staging without exposing real PII.

Reference: [How to clone shipment from Production](#)

63. Use the Postman API 'Clone shipment for other client'.
64. Select the Prod environment and the admin token.
65. Input the list of shipment IDs you want to clone from Prod.
66. Click 'Save and Download' — a CSV is downloaded.
67. Use that CSV to upload shipments in Staging Client web.

11.4 Route the Shipment (Routing Web App)

Reference: [How to Route Using the Routing Web App](#)

Precondition

- Client has submitted a CSV file or single shipment to Jitsu.

Steps

68. Open the Routing web: <https://routing.staging.gojitsu.com/>
69. Click 'Create Problem'.
70. Choose 'From Jobs' (if submitted as CSV from Client app).
71. Select the uploaded Jobs.
72. Select the warehouse whose region matches the submitted CSV.
73. Click Save.
74. Click 'Select stops' in the header, draw a region on the map, name it, and click Save.
75. Click 'Add Region Settings' → Save. Pick the region, choose a prefix from the dropdown, type something into 'Start With'.
76. Click 'Split Problem'.
77. Click 'Show Left Panel'.
78. In each sub-problem, click 'Vehicles' → 'Add New Vehicle' → Save. Set Capacity and Available (number of vehicles).
79. Once all regions are set, click 'Reroute All'.
80. Wait until 'Select solution' appears, click Select.
81. Click the Solution number to view the solution details. This generates Assignments — sets of shipments grouped along an optimized route.

11.5 Sign Up & Approve a Driver

Reference: [How to sign up driver in Driver App](#)

Steps

82. Sign up a driver through the Driver App following the standard flow.
83. Get the driver's access token via the [OAuth] Login API in Postman.
84. In Staging, after the driver account is created, call 7.[Reg]Simulate fcra approval to auto-approve the driver (uses the driver's token).



STAGING ONLY

The 'Simulate fcra approval' shortcut is only for Staging. On Production, drivers must go through the real background check process.

Simulate Turn Status (Background Check)

Reference: [How to simulate the turn status after submitting background check](#)

Turn API docs: [Turn Public API](#)

In Postman, use the API 'Outcome received' to simulate the turn state. After the driver submits BG, data is saved in the driver_registration_record collection in MongoDB:

- turn_id → exo_id
- worker_id → exo_transaction_id
- reference_id → reference_id
- state → from Turn API documentation

11.6 Create Advance Booking Session

Reference: [How to create the Advance Booking Session](#)

Steps

85. Open the Dashboard.
86. Click the 'Forecast' button.
87. Select the region you want to create a Booking Session for — all templates of that region appear.
88. Select any template — the Forecast detail page is shown.
89. Click 'Create Booking Session' — the dialog appears.

Fields to fill

- Target date — the date when the driver has to deliver.
- Booking start time — when the driver can start booking tickets from this BS.
- Template — choose the BS template.

90. Click Create — the Create Booking Session pop-up appears.
91. Click 'Go to Booking Session' to view the BS details.

Demo video: [Create Direct Booking from Booking session — Demo](#)

11.7 Create Schedule / Direct Booking

Reference: [How to create the Schedule/Direct booking](#)

Steps

92. Open the Schedule page (Staging): <https://dispatch.staging.gojitsu.com/schedule>
93. Click 'New Schedule'.

Fields

- Region — region for which the Schedule is created.
- Date — target delivery date.
- Name — Schedule name visible in the Driver App and Dispatch.
- Max Reservation — number of routes a driver can book in this Schedule.

94. Click Save.
95. Add routes and drivers (one by one) to the created Schedule.

11.8 Create DSP Account & DSP Driver

Create a DSP Account

Reference: [How to create the DSP account](#)

96. Open the Dashboard and log in as admin.
97. Open the left menu (hamburger icon).
98. Click 'DSP' — go to the DSP page.
99. Click 'New' to add a DSP courier. (Don't forget the Alias field — it is required.)
100. Fill in valid DSP info and click ADD — the account is created and listed.
101. Log in via the DSP Portal:
<https://oauth2.staging.gojitsu.com/login?next=https://dsp.staging.gojitsu.com/>
102. Set up payment for the DSP after creation.

Create a DSP Driver

Reference: [How to create the DSP Driver account](#)

103. First, create a DSP account (see above).
104. Go to the DSP Portal: <https://dsp.staging.gojitsu.com/> and log in with the DSP account.
105. Select the Drivers tab → click 'New' to add a new driver.
106. Fill in the driver's data.
107. Log in to the Dashboard with admin to approve the driver:
<https://dashboard.staging.gojitsu.com/dsp>
108. Open the left menu → DSP → DSP couriers → find the new driver → approve.

11.9 Scan All Shipments of a Solution Automatically (Postman)

Reference: [How to scan all shipments of a solution automatically in Postman](#)

This automates inbound scanning when you have a solution ID. Example: solution ID 56433.

Postman Setup

109. Create a new folder, e.g. 'scan all shipments in solution'.
110. Add a GET request inside it:
`{{routing_app_url}}/routing/solutions/{{solutionId}}/assignments`

Tests tab script:

```
var jsonData = pm.response.json(); var listAssignments =
jsonData.assignments.map(item => item.id);
pm.environment.set("assignmentIds", JSON.stringify(listAssignments))
pm.environment.set("shipmentIds", "[]")
```

111. Add a request named 'getShipment':
`{{dataorch_url}}/assignments/{{assignmentId}}/detail`

Pre-request Script:

```
var list = JSON.parse(pm.environment.get("assignmentIds"));
pm.environment.set("assignmentId", list.shift());
pm.environment.set("assignmentIds", JSON.stringify(list));
```

Tests Script (chains to next request if more assignments remain):

```
var jsonData = pm.response.json(); let shipments =
jsonData.shipments.map(item => item.id); let shipmentIds
= JSON.parse(pm.environment.get("shipmentIds"));
shipmentIds.push(...shipments); pm.environment.set("shipmentIds",
JSON.stringify(shipmentIds)); let assignmentIds =
JSON.parse(pm.environment.get("assignmentIds")); if (assignmentIds.length >
0) postman.setNextRequest("getShipment");
```

112. Add a request named 'scanShipment': `{{inbound_api_url}}/info/{{shipmentID}}`

Pre-request:

```
var shipmentList = JSON.parse(pm.environment.get("shipmentIds"));
pm.environment.set("shipmentID", shipmentList.shift());
pm.environment.set("shipmentIds", JSON.stringify(shipmentList));
```

Tests:

```
pm.test("Status code is 200", function ()
{ pm.response.to.have.status(200); }); let shipmentList =
JSON.parse(pm.environment.get("shipmentIds")); if (shipmentList.length > 0)
postman.setNextRequest("scanShipment");
```

113. Right-click the folder → Run Collection.

11.10 Test Dimensions (Warehouse App)

Reference: [How to test Dimensions](#)

Actual flow: Warehouse scans the shipment → a 'Measure' label is generated → staff scans to measure the box in the WH app → scans a second time to receive the box.

Configuration

- sort_center collection in MongoDB has 'enable_measure' per sort center (true/false).
- dimension_config is also configured per sort_center.
- Min/Max samples are configured in the drl table by key 'calculate_consistency_score'.

Record Dimensions Using the Warehouse App

Reference: [How to record dimensions using the Warehouse App](#)

In production a vMeasure machine reads the box automatically. For testing, simulate it with the oz_cli tool.

114. Obtain a build of oz_cli (e.g. from the gojitsucom/oz repo).
115. Install oz_cli_config.json (see the oz repo README).
116. Run: oz measurement_machine connect -w 193 -n "Test Machine" (-w = warehouse ID, -n = display name).
117. Launch the Warehouse App on a TC26 Zebra scanner → side menu → Measure Parcels → select the test machine.
118. Scan a shipment ID QR code.

Verification

Measurement records are saved in MongoDB collection 'measurement_obligations'. Look up the shipment_id you just measured to confirm the value.

11.11 Test Labels

Reference: [How to execute test label](#)

Label version is decided by config keys in MongoDB. The selection rule is summarized below.

Has parcel?	parcel_label_v ersion + label_version	parcel_label_v ersion only	label_version only	No config
Yes (has parcel)	parcel_label_ve rsion	parcel_label_ver sion	label_version	postgres config
No (no parcel)	label_version	postgres config	label_version	postgres config

Download Label — Public API (Client app, single shipment)

- PDF — get version from Postgres first; if shipment has parcels, use mongo 'parcel_label_version' instead.
- ZPL — get version from mongo 'zpl-label-version'; if missing, return DEFAULT (hardcoded).

Worker (PDF, multiple shipments)

- LabelClientShipmentWorker: mongo 'parcel_label_version' (high priority) → 'label_version' (medium).
- LabelJobWorker: 'presets.version' → fallback to Postgres.

Data Orchestra (PDF) — Dispatch App

- Shipment has parcels → 'parcel_label_version'.
- Shipment has no parcels → 'zpl-label-version' → fallback to Postgres.

Ares Label

Clients currently using Ares:

Client ID	Client Name
18	Mar
201	Whiplash
164	The Farmers Dog
4424	ShipHero
176	Splendid Spoon
76	Hungryroot

Client ID	Client Name
99	Dinnerly
157	Flexport
5171	Ollie Pets
5104	Wildgrain
5085	Feed
4526	Tangelo

To test Ares: use API Axlehire-Ares-Shippo > Label Purchase in Postman (public token).

11.12 Test the Weekly Payment System

Reference: [How to Test the Weekly Payment System](#)

Use the driver_payment worker on the Schedules page.

URL: <https://dashboard.staging.gojitsu.com/schedules>

TIP

Change the worker's 'Firing Time' to trigger a payout for the current date — saves you waiting for the weekly cron.

Body for payment by each driver

Fields:

- look_back_days — number of days to look back.
- driver_ids — driver IDs to pay out.
- driver_payment_method — payment method (e.g. BRANCH_APP).

Example body:

```
{  "type": "CUSTOM_METHOD",  "driver_payment_method": "BRANCH_APP",  "look_back_days": "2",  "driver_ids": "7873" }
```

11.13 Test Disposable Shipments

Reference: [How to test disposable in Staging](#)

Use RabbitMQ + the shipment_disposable worker. Note: change the firing time to match your disposable window.

Example worker config:

```
{  "attributes": {    "shipment_status": "DROPOFF_FAILED, PICKUP_FAILED, GEOCODE_FAILED, GEOCODED, RETURN_SUCCEEDED, PICKUP_SUCCEEDED, RESCHEDULED, ASSIGNED, INVALID_ADDRESS"  },  "type": "shipment_disposable",  "name": "SHIPMENT DISPOSABLE",  "active": true,  "schedule": "0 5 * ? * * *",  "time_zone": "America/Los_Angeles",  "snap_to": "hourly",  "start": "-2h",  "end": "0h" }
```

Update Vehicle Insurance & Registration in the Driver App

Reference: [How to update Vehicle Insurance / Registration info](#)

To update the current vehicle, edit the Primary Vehicle.

Vehicle Insurance

119. Tap Profile → Vehicle Information.
120. In the Vehicle Insurance Photo section, tap Edit.
121. Input the OTP sent to driver email/phone.
122. Edit the info and tap Save.

Vehicle Registration Record

123. Same flow as above, but choose the Vehicle Registration Record photo section.

12. Client Web — Functional Reference

Reference: [Client session training](#)

Production URL: <https://client.gojitsu.com/>

Shipments

- Shipment list — shows all today's shipments by default. Single region → map auto-zooms in. Multiple regions → map zooms out, user can pan/zoom.
- Filter shipments.
- Download CSV / Download PDF.
- Shipment breakdown.
- Shipment detail — view POD & signature, send feedback, download full label, track.

Search

Search by Customer name, tracking code, email, phone, address — filter by Brand or Region.

Batch

- Upload shipment file → Finalize.
- Delete job.
- Update draft shipment address.

Create Single Shipment

- Inline form on Client web.

Reports

Client can download 4 report types: Shipments, Inbound Exception Report, Outbound Exception Report, Return Shipment Report.

13. Recipient Web — Functional Reference

Reference: [Recipient session training](#)

Production URL: <https://recipient.gojitsu.com/>

Manage Delivery

Only visible while shipment is not completed. Allows the recipient to make changes to the upcoming delivery.

Address Questionnaire

Asks the recipient about their address type. If 'single house' is selected, follow-up questions change accordingly.

Edit Address

Recipient can update Address 1, Address 2, etc.

Codes

Call-in code, building code, entrance/gate code, etc.

Delivery Instructions

Location Pins & Preferred Drop-off

Recipient pins a preferred drop-off location and uploads a photo to help the driver.

Feedback

- Upload Preferred Delivery Photo.
- Rate POD (like / dislike).
- Send feedback message.
- Tip the driver.

14. Quick Reference — Useful Links

All the Confluence pages this training was synthesized from, plus the most important external links.

Main Training Hub

- [Training Document \(Confluence\)](#)

Overview Pages

- [Overall all systems](#)
- [Main function](#)
- [Client Service Type](#)

QA Process

- [QA Process](#)
- [Important Note for QA team in Prod](#)
- [Test Case Template](#)

How-To Guides — E2E Flow

- [StrongDM — Database & Tools Access Setup](#)
- [How to create Client](#)
- [How to submit a shipment from Client web](#)
- [How to clone shipment from Production](#)
- [How to Route Using the Routing Web App](#)
- [How to sign up driver in Driver App](#)
- [Simulate turn status \(background check\)](#)
- [How to create the DSP account](#)
- [How to create the DSP Driver account](#)
- [How to create the Advance Booking Session](#)
- [How to create the Schedule/Direct booking](#)
- [Scan all shipments automatically in Postman](#)
- [How to test Dimensions](#)
- [Record dimensions using the Warehouse App](#)
- [How to execute test label](#)
- [How to Test the Weekly Payment System](#)
- [How to test disposable in Staging](#)
- [Update Vehicle Insurance / Registration info](#)

Training Demos

- [Booking session training](#)
- [Create Direct Booking from Booking session — Demo](#)
- [Client session training](#)
- [Recipient session training](#)

External Resources

- [Turn Public API documentation](#)

End of Training Document — Welcome to the Jitsu QA Team! 🚀

When in doubt, search Confluence first, then ask your QA lead.